Knowledge Representation

Slides by:

Jiawei Han, Micheline Kamber, and Jian Pei

University of Illinois at Urbana-Champaign &

Simon Fraser University

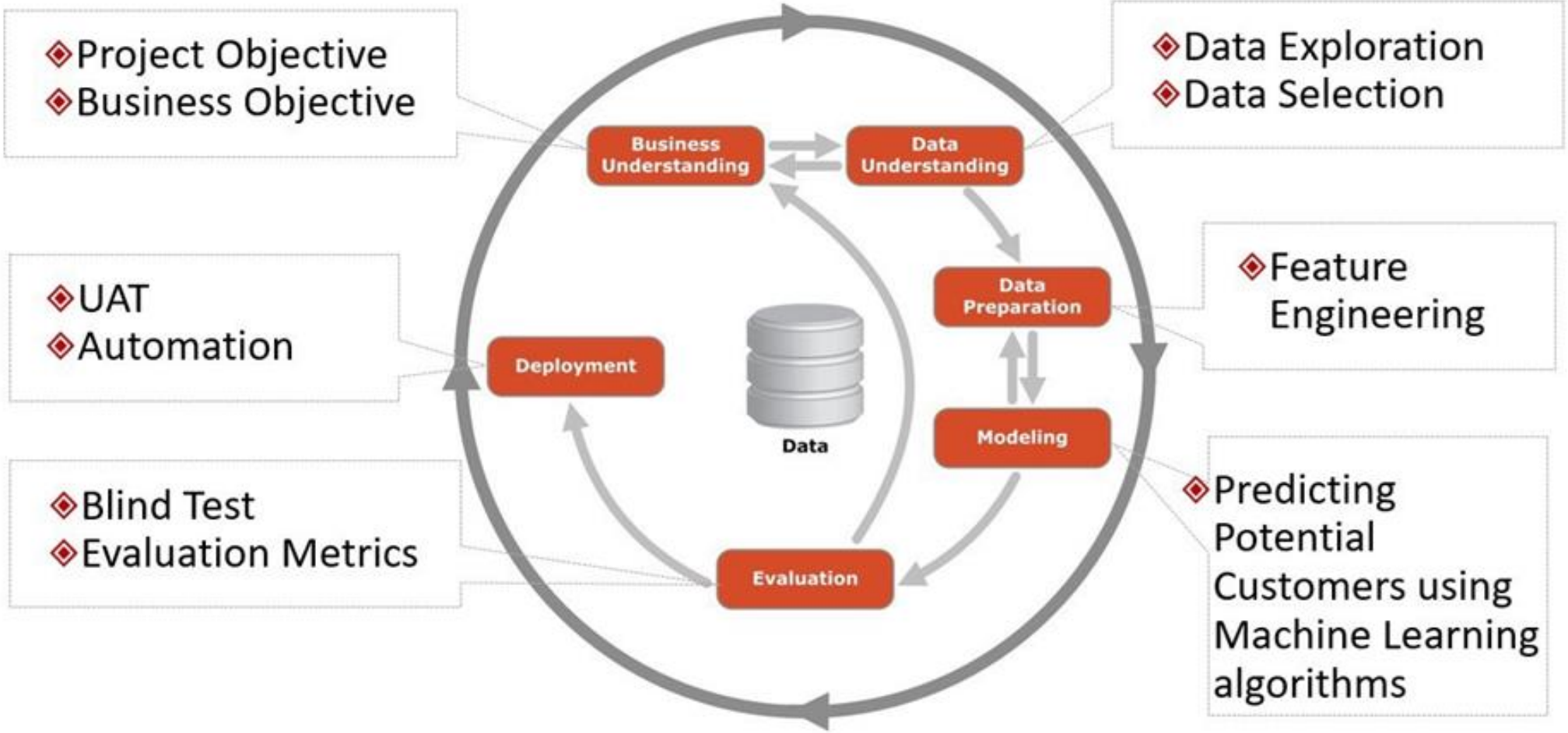
©2023 Han, Kamber & Pei. All rights reserved.

The slides have been modified and rearranged by G.A. Putri Saptawati for the purpose of course IF4041 Data Mining

Metodologi Pengembangan: CRISP-DM

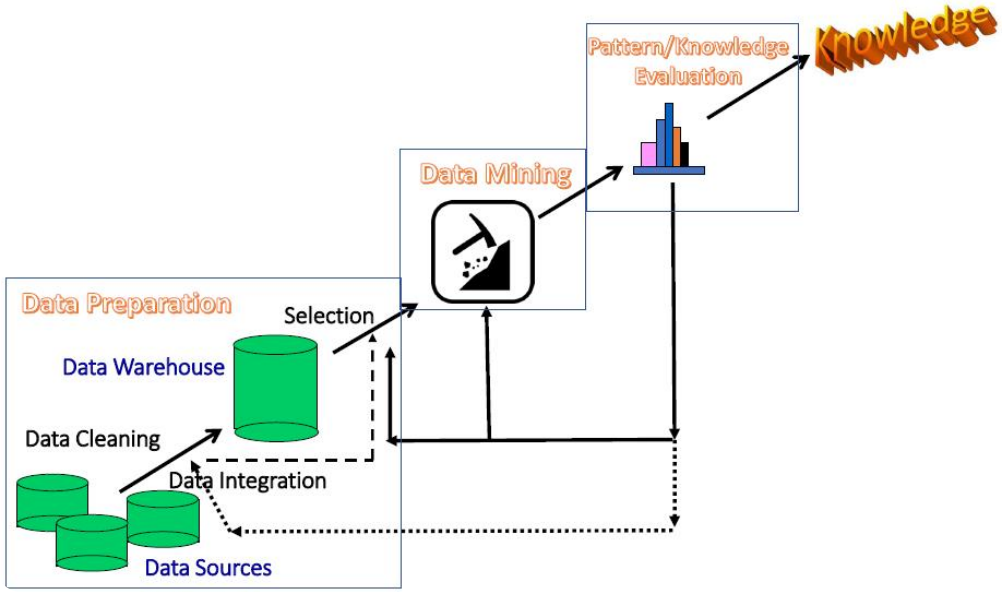
Cross-Industry Standard Process for Data Mining

https://medium.com/@sumit.yg/analyzing-seattle-s-airbnb-listings-data-49abdc0977c8



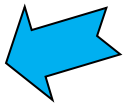
2

Data Mining: An Essential Step in Knowledge Discovery



- Knowledge Discovery Process
  - Data preparation
    - Data cleaning
    - Data integration
    - Data transformation
    - Data selection
  - Data mining
  - Pattern/model evaluation
  - Knowledge presentation

Output : Knowledge Representation

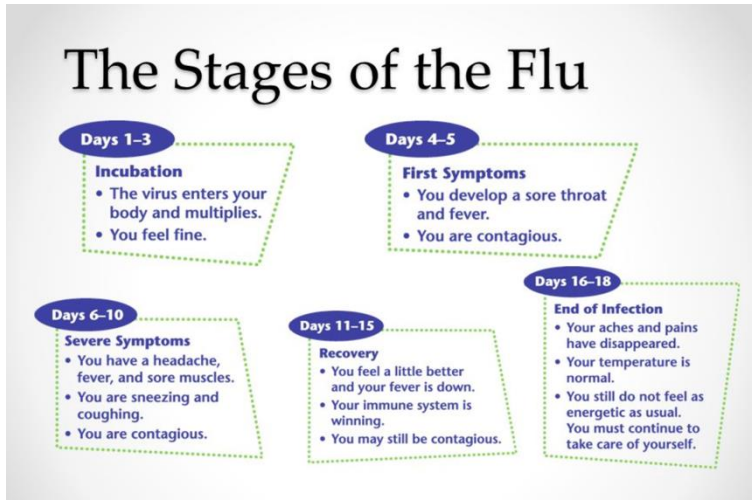
- Pattern Mining : Basic Concepts 
- Classification: Basic Concepts
- Clustering : Basic Concepts
- Outlier Detection : Basic Concepts

What Are Patterns?

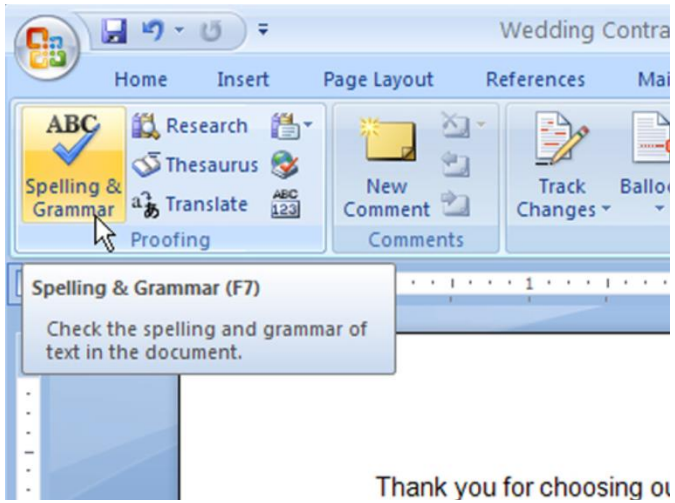
- What are patterns?
  - Patterns: A set of items, subsequences, or substructures that occur frequently together (or strongly correlated) in a data set
  - Patterns represent intrinsic and important properties of datasets



Frequent item set



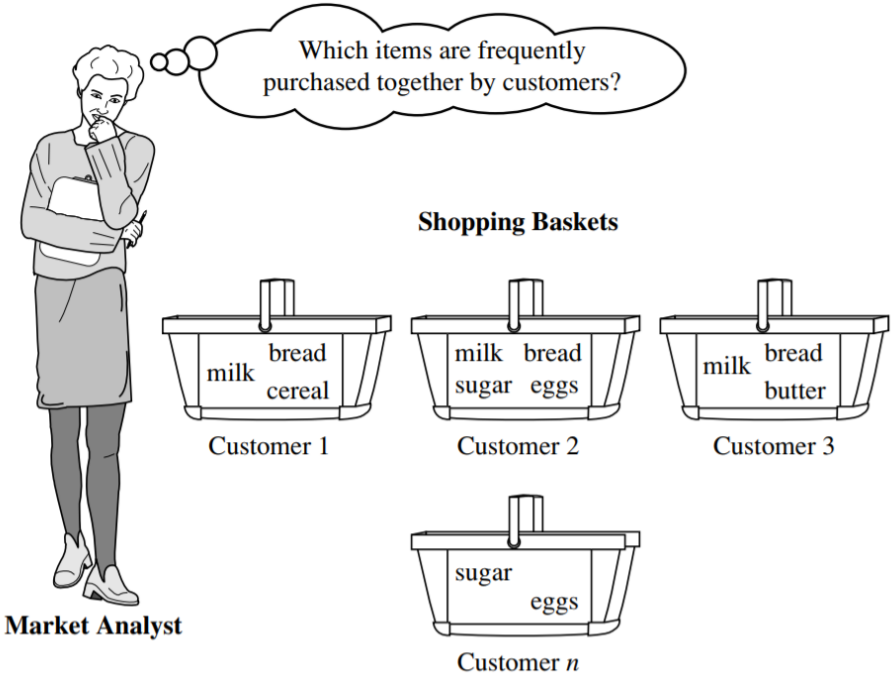
Frequent sequences



Frequent structures

What Is Pattern Discovery?

- Pattern discovery: Uncovering patterns from massive data sets
- It can answer questions such as:
  - What products were often purchased together?
  - What are the subsequent purchases after buying an iPad?



Pattern Discovery: Why Is It Important?

- Foundation for many essential data mining tasks
  - Association, correlation, and causality analysis
  - Mining **sequential**, structural (e.g., sub-graph) patterns
  - **Classification**: Discriminative pattern-based analysis
  - **Cluster** analysis: Pattern-based subspace clustering
- Broad applications
  - Market basket analysis, cross-marketing, catalog design, sale campaign analysis, Web log analysis, biological sequence analysis
  - Many types of data: spatiotemporal, multimedia, time-series, and stream data

Basic Concepts: Transactional Database

- Transactional Database (TDB)
  - Each transaction is associated with an identifier, called a TID.
  - May also have counts associated with each item sold

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Output : Knowledge Representation

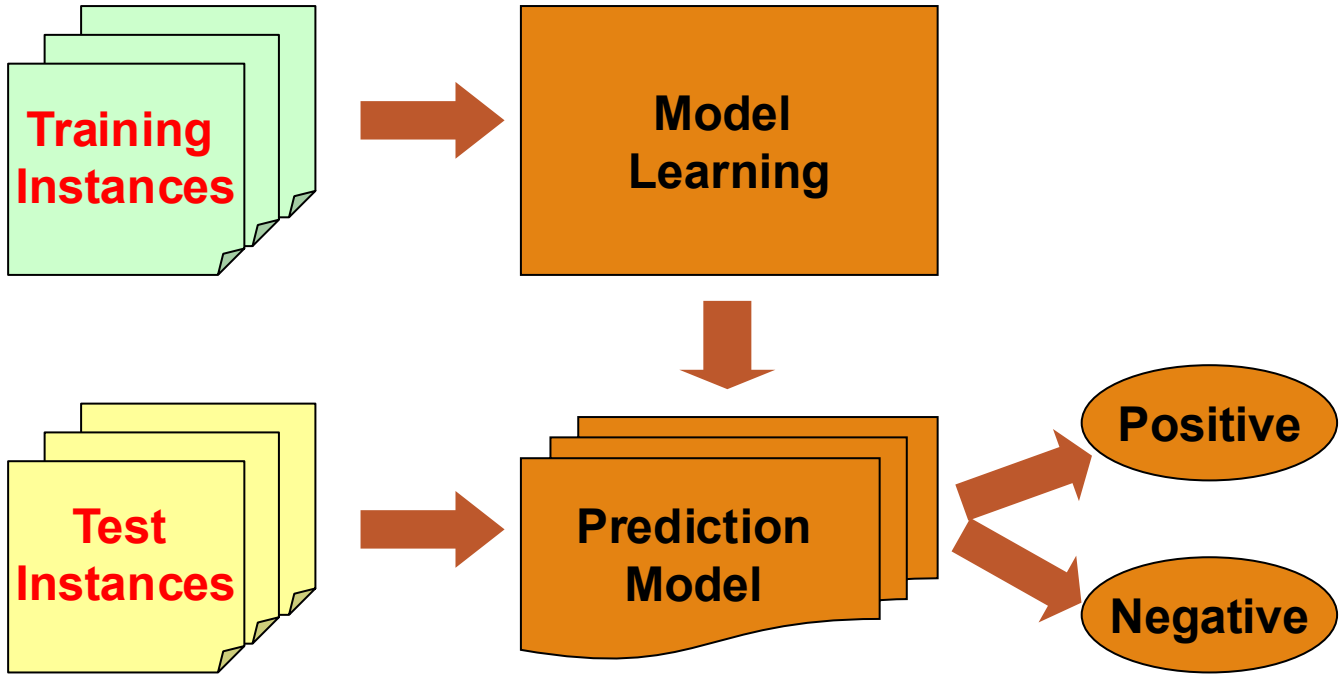
- Pattern Mining : Basic Concepts
- Classification: Basic Concepts 
- Clustering : Basic Concepts
- Outlier Detection : Basic Concepts

Supervised vs. Unsupervised Learning (1)

- Supervised learning (classification)
  - Supervision: The training data such as observations or measurements are accompanied by **labels** indicating the classes which they belong to
  - New data is classified based on the models built from the training set

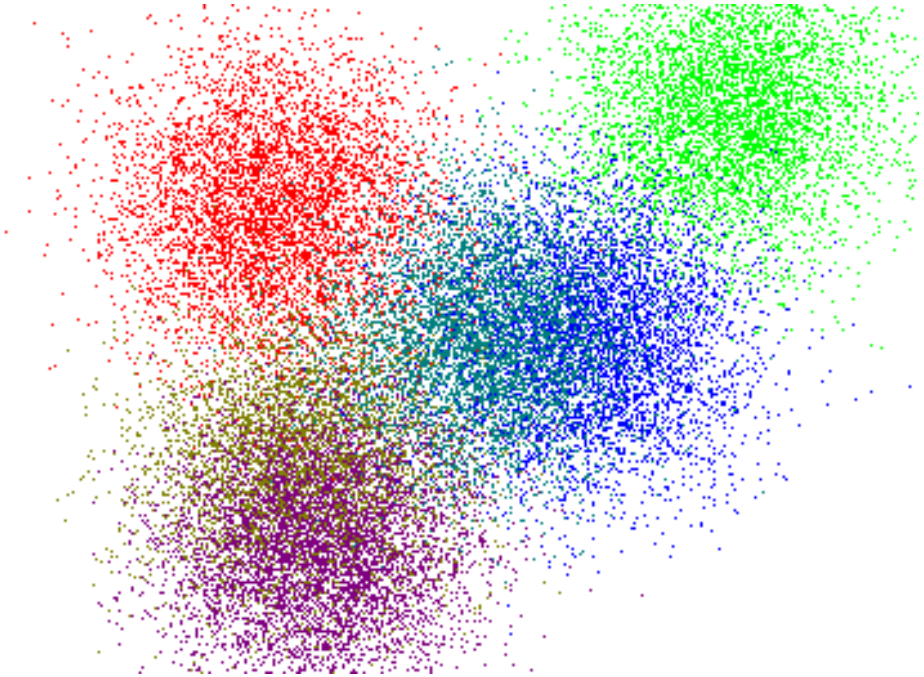
Training Data with class label:

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No



Supervised vs. Unsupervised Learning (2)

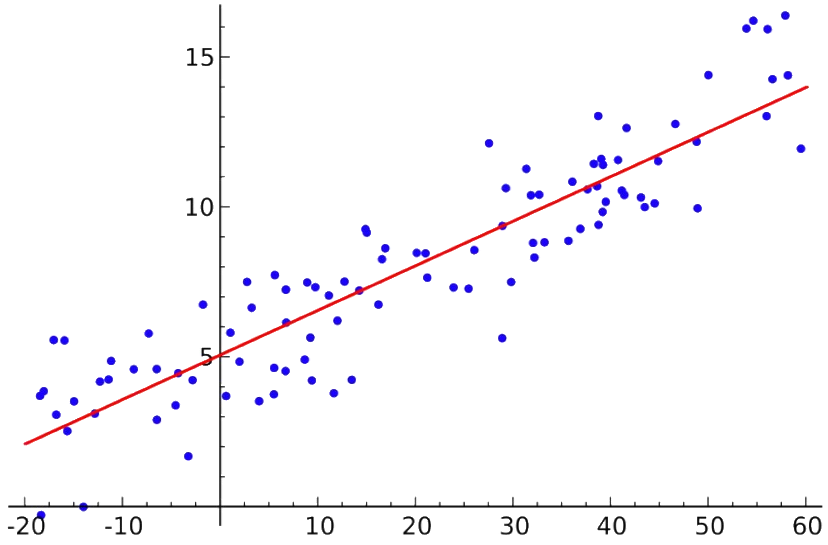
- Unsupervised learning (clustering)
  - The class labels of training data are **unknown**
  - Given a set of observations or measurements, establish the possible existence of classes or clusters in the data





Prediction Problems: Classification vs. Numeric Prediction

- Classification
  - Predict **categorical class labels** (discrete or nominal)
  - Construct a model based on the training set and the **class labels** (the values in a classifying attribute) and use it in classifying new data
- Numeric prediction
  - Model **continuous-valued functions** (i.e., predict unknown or missing values)

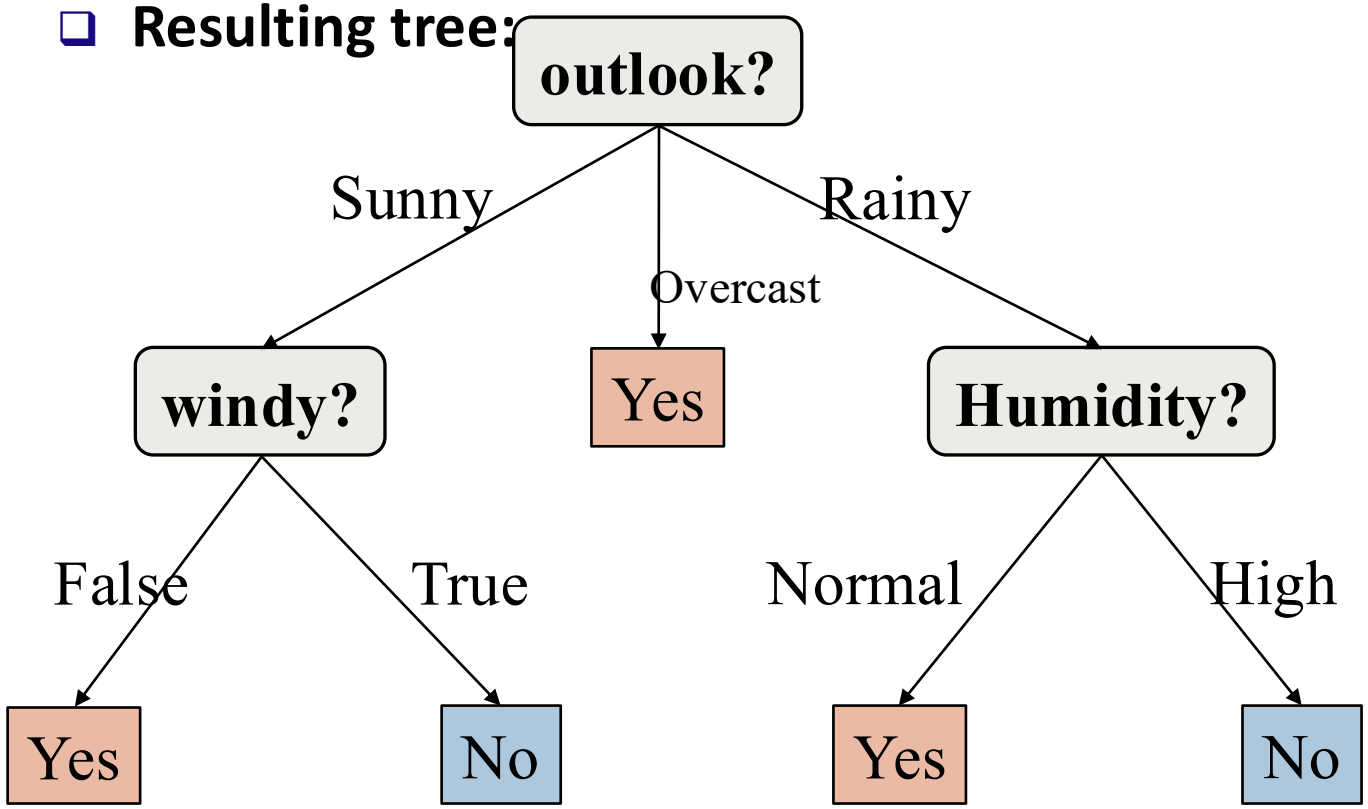


Classification—Model Construction, Validation and Testing

- **Model Construction and Training**
  - Model: Represented as decision trees, rules, mathematical formulas, or other forms
  - Assumption: Each sample belongs to a predefined class /**class label**
  - Training Set: The set of samples used for model construction

Decision Tree Induction: An Example

- **Decision tree construction**:
  - A top-down, recursive, divide-and-conquer process
- **Resulting tree:**



Training data set: Play Golf?

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

https://www.saedsayad.com/decision_tree.htm

Decision Tree Induction: Algorithm

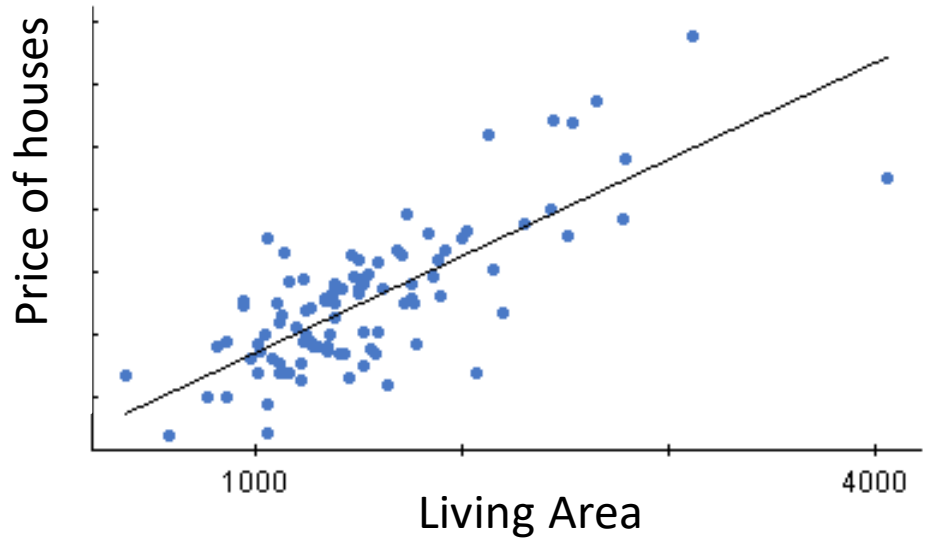
- Basic algorithm
  - Tree is constructed in a **top-down, recursive, divide-and-conquer manner**
  - At start, all the training examples are at the root
  - Examples are partitioned recursively based on selected attributes
  - On each node, attributes are selected based on the training examples on that node, and a heuristic or statistical measure (e.g., **information gain, Gini index**)

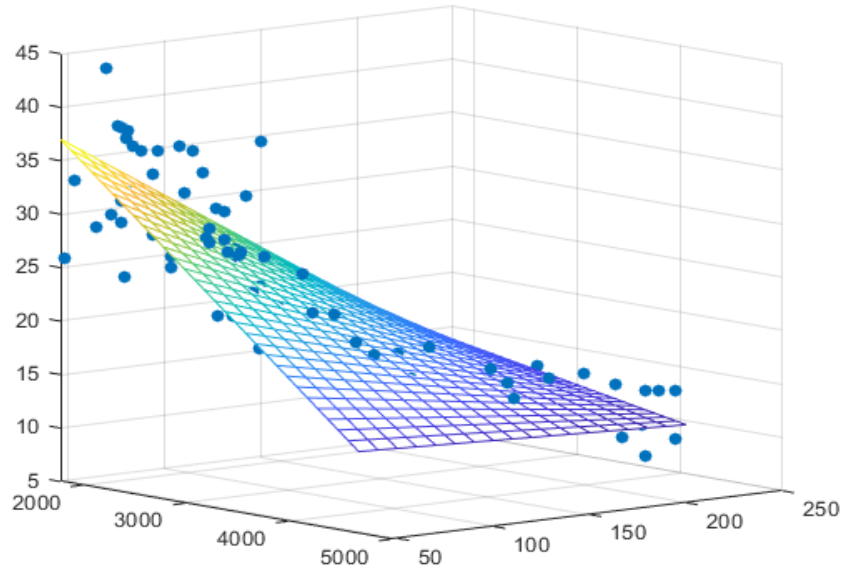
Decision Tree Induction: Algorithm

- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - There are no remaining attributes for further partitioning
  - There are no samples left
- Prediction
  - **Majority voting** is employed for classifying the leaf

Linear Regression Problem: Example

- Mapping from independent attributes to **continuous value**: x => y
- {living area} => Price of the house
- {college; major; GPA} => Future Income





Linear Regression Problem: Model

- Linear regression
  - Data: n independent objects
    - Observed Value: $y_i, i = 1,2,3, \cdots, n$
    - p-dimensional attributes: $x_i = \left(x_{i1}, x_{i2}, \cdots, x_{ip}\right)^T,\ i = 1,2,3 \cdots, n$
  - Model:
    - Weight vector: $w = \left(w_1, w_2, \cdots, w_p\right)$
    - $y_i = w^T x_i + b$
  - The weight vector w and bias b are the model parameter learnt by data

Linear Regression Model: Solution

- Least Square Method
  - Cost / Loss Function: $\mathrm{L}(w,b) = \Sigma_{i=1}^{n}(y_i - wx_i - b)^2$
  - Optimization Goal: $\underset{(w,b)}{\operatorname{argmin}}\ \mathrm{L}(w,b) = \Sigma_{i=1}^{n}(y_i - wx_i - b)^2$
  - Closed-form solution:
    - $w = \frac{\Sigma_{i=1}^{n} x_i(y_i - \bar{y})}{\Sigma_{i=1}^{n} x_i^2 - n(\Sigma_{i=1}^{n} x_i)^2}$ $\qquad b = \frac{1}{n}\Sigma_{i=1}^{n}(y_i - wx_i)$

Using IF-THEN Rules for Classification

- Represent the knowledge in the form of **IF-THEN** rules

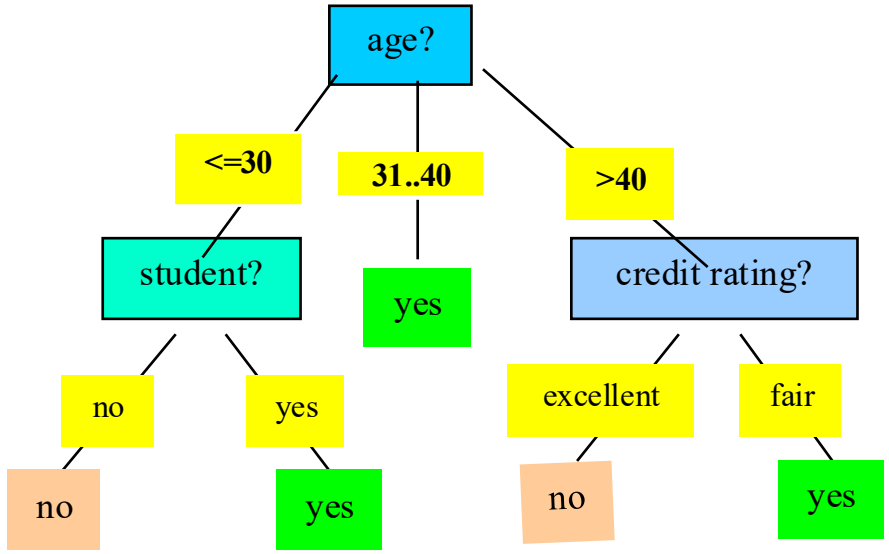
  R_1: IF *age* = youth AND *student* = yes THEN *buys_computer* = yes

- Assessment of a rule: *coverage* and *accuracy*
  - coverage(R_1) = ratio of tuples covered by **the condition of** R_1 (THEN-part is not important for this)
  - accuracy(R_1) = ratio of tuples correctly classified by R_1 in the covered ones (both IF-part and THEN-part counts)

- If more than one rule are triggered, need **conflict resolution**
  - **Size ordering**: assign the highest priority to the triggering rules that has the "toughest" requirement (i.e., with the *most attribute tests*)
  - **Class-based ordering**: decreasing order of *prevalence or misclassification cost per class*
  - **Rule-based ordering** (**decision list**): rules are organized into one long priority list, according to some measure of rule quality or by experts

Rule Extraction from a Decision Tree

- Rules are *easier to understand* than large trees
- One rule is created *for each path* from the root to a leaf
- Each attribute-value pair along a path forms a conjunction: the leaf holds the class prediction
- Rules are mutually exclusive and exhaustive



- Example: Rule extraction from our *buys_computer* decision-tree

IF *age* = young AND *student* = *no* THEN *buys_computer* = *no*

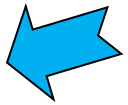
IF *age* = young AND *student* = *yes* THEN *buys_computer* = *yes*

IF *age* = mid-age THEN *buys_computer* = *yes*

IF *age* = old AND *credit_rating* = *excellent* THEN *buys_computer* = *no*

IF *age* = old AND *credit_rating* = *fair* THEN *buys_computer* = *yes*

Output : Knowledge Representation

- Pattern Mining : Basic Concepts
- Classification: Basic Concepts
- Clustering : Basic Concepts 
- Outlier Detection : Basic Concepts

What Is Cluster Analysis?

- What is a cluster?
  - A cluster is a collection of data objects which are
    - Similar (or related) to one another within the same group (i.e., cluster)
    - Dissimilar (or unrelated) to the objects in other groups (i.e., clusters)
- Cluster analysis (or clustering, data segmentation, …)
  - Given a set of data points, partition them into a set of groups (i.e., clusters) which are as similar as possible
- Cluster analysis is unsupervised learning (i.e., no predefined classes)
  - This contrasts with classification (i.e., supervised learning)
- Typical ways to use/apply cluster analysis
  - As a stand-alone tool to get insight into data distribution, or
  - As a preprocessing (or intermediate) step for other algorithms

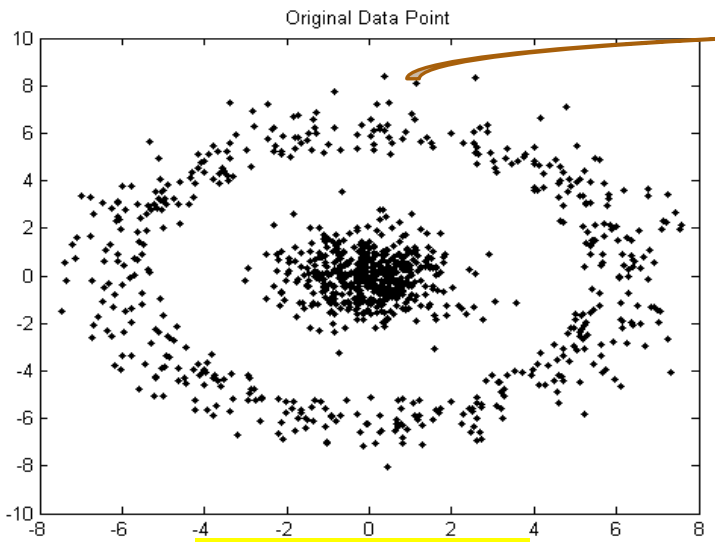
Cluster Analysis: Applications

- A key intermediate step for other data mining tasks
  - Generating a compact summary of data for classification, pattern discovery, hypothesis generation and testing, etc.
  - Outlier detection: Outliers—those “far away” from any cluster
- Data summarization, compression, and reduction
  - Ex. Image processing: Vector quantization
- Collaborative filtering, recommendation systems, or customer segmentation
  - Find like-minded users or similar products
- Dynamic trend detection
  - Clustering stream data and detecting trends and patterns
- Multimedia data analysis, biological data analysis and social network analysis
  - Ex. Clustering images or video/audio clips, gene/protein sequences, etc.

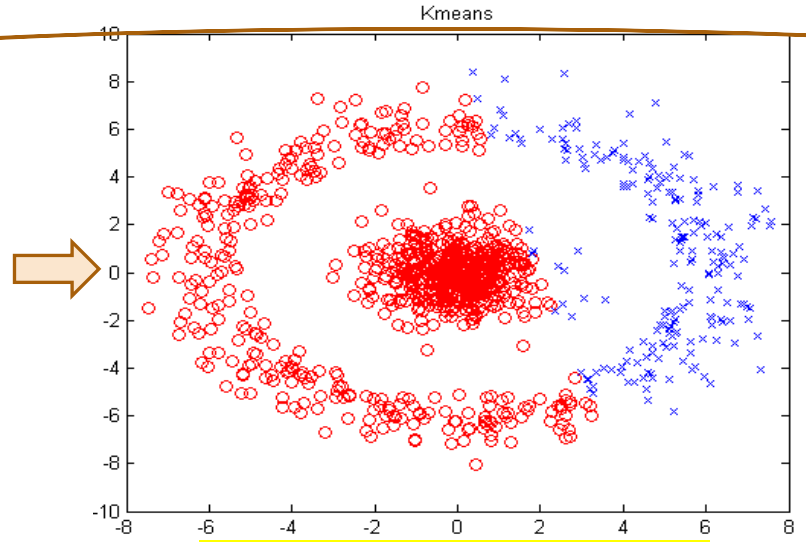
Considerations for Cluster Analysis

- Partitioning criteria
  - Single level vs. hierarchical partitioning (often, multi-level hierarchical partitioning is desirable, e.g., grouping topical terms)
- Separation of clusters
  - Exclusive (e.g., one customer belongs to only one region) vs. non-exclusive (e.g., one document may belong to more than one class)
- Similarity measure
  - Distance-based (e.g., Euclidean, road network, vector) vs. connectivity-based (e.g., density or contiguity)
- Clustering space
  - Full space (often when low dimensional) vs. subspaces (often in high-dimensional clustering)

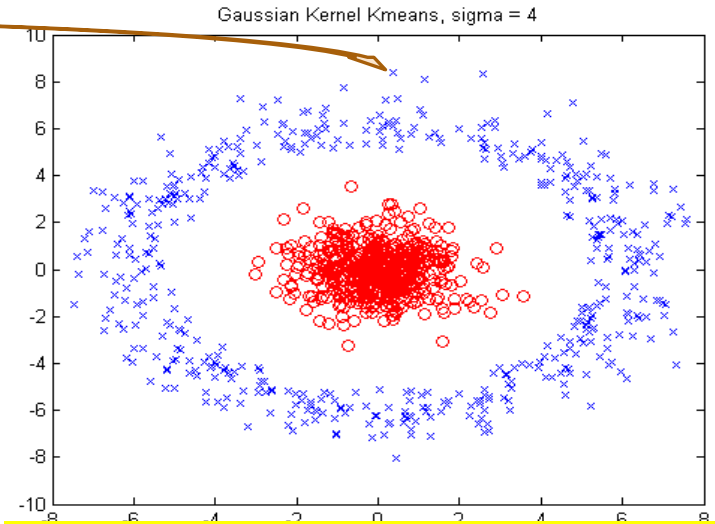
Example: Kernel K-Means Clustering



The original data set



The result of K-Means clustering

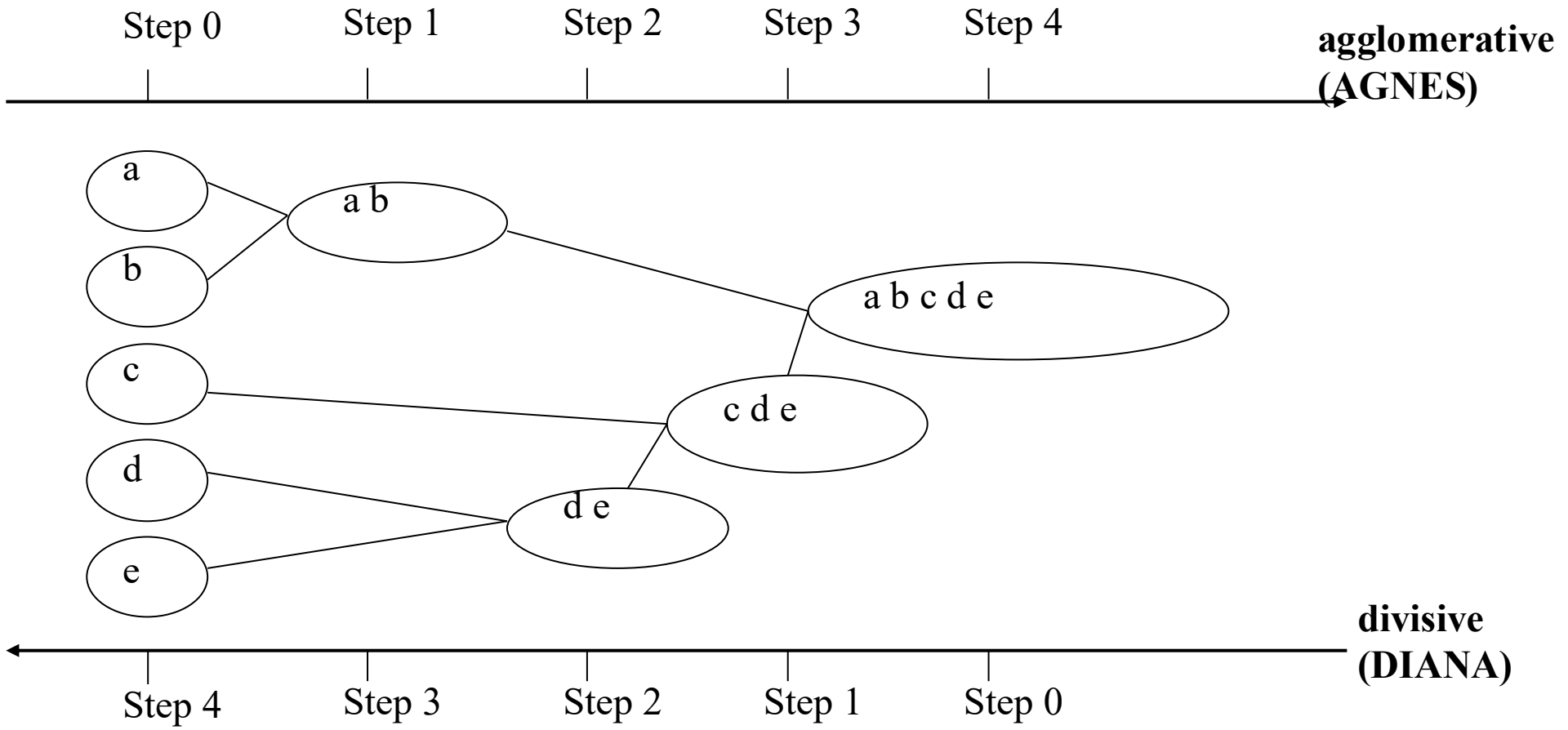


The result of Gaussian Kernel K-Means clustering

Hierarchical Clustering: Basic Concepts

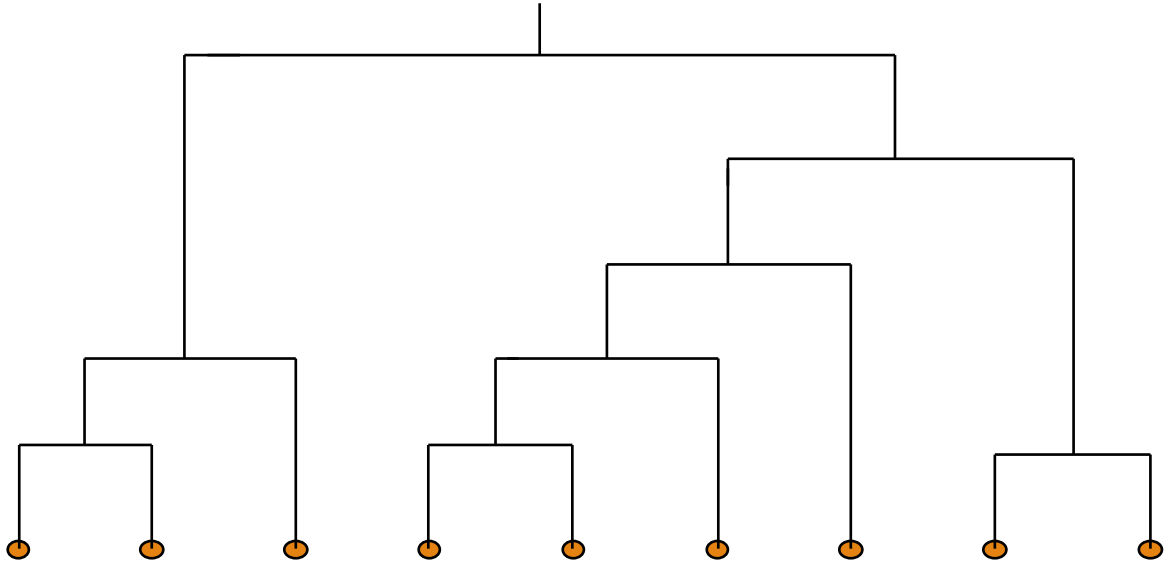
- Hierarchical clustering
  - Generate a clustering hierarchy (drawn as a dendrogram)
  - Not required to specify K, the number of clusters
  - More deterministic
  - No iterative refinement
- Two categories of algorithms
  - Agglomerative: Start with singleton clusters, continuously merge two clusters at a time to build a bottom-up hierarchy of clusters
  - Divisive: Start with a huge macro-cluster, split it continuously into two groups, generating a top-down hierarchy of clusters

Agglomerative vs. Divisive Clustering



Dendrogram: How Clusters are Merged

- Dendrogram: Decompose a set of data objects into a tree of clusters by multi-level nested partitioning
- A clustering of the data objects is obtained by cutting the dendrogram at the desired level, then each connected component forms a cluster



Hierarchical clustering generates a dendrogram (a hierarchy of clusters)

Density-Based Clustering Methods

- Clustering based on density (a local cluster criterion), such as density-connected points
- Major features:
  - Discover clusters of arbitrary shape
  - Handle noise
  - One scan (only examine the local region to justify density)
  - Need density parameters as termination condition
- Several interesting studies:
  - DBSCAN: Ester, et al. (KDD'96)
  - OPTICS: Ankerst, et al (SIGMOD'99)
  - DENCLUE: Hinneburg & D. Keim (KDD'98)
  - CLIQUE: Agrawal, et al. (SIGMOD'98) (also, grid-based)

Example of DBSCAN

Figure 8. DBScan results for DS1 with MinPts at 4 and Eps at (a) 0.5 and (b) 0.4.

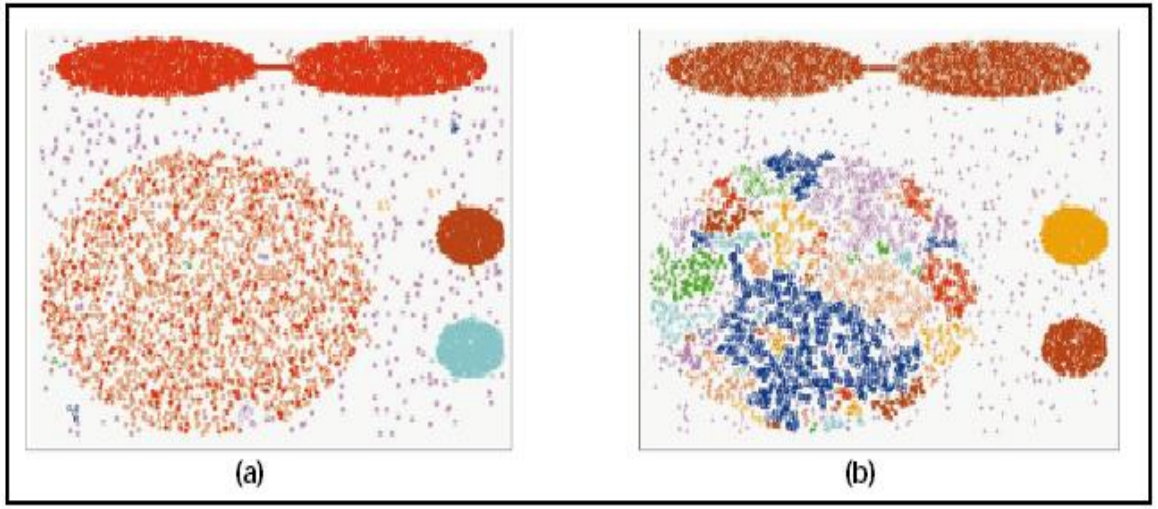
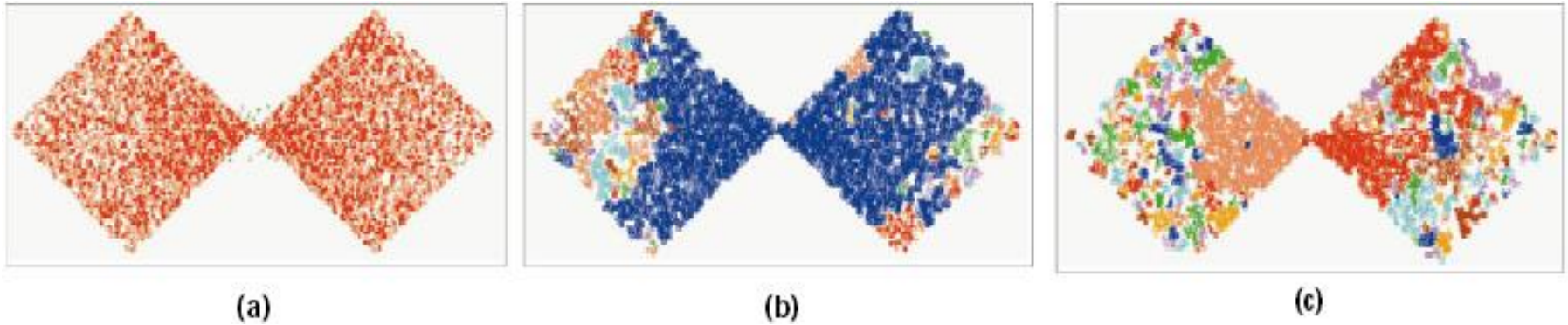


Figure 9. DBScan results for DS2 with MinPts at 4 and Eps at (a) 5.0, (b) 3.5, and (c) 3.0.



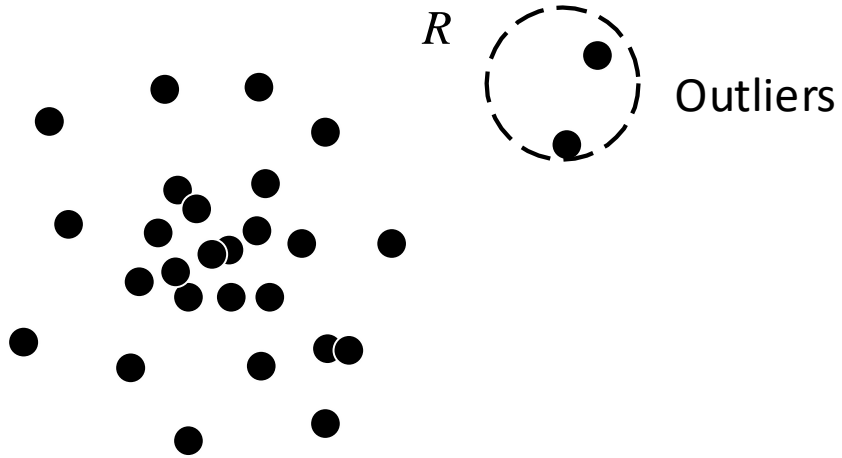
Ack. Figures from G. Karypis, E.-H. Han, and V. Kumar, *COMPUTER*, 32(8), 1999

Output : Knowledge Representation

- Pattern Mining : Basic Concepts
- Classification: Basic Concepts
- Clustering : Basic Concepts
- Outlier Detection : Basic Concepts 

What Are Outliers?

- Assume that a given statistical process is used to generate a set of data objects
- An outlier is a data object that deviates significantly from the rest of the objects, as if it were generated by a different mechanism



33

Outlier Detection versus Clustering

- Two highly related but different tasks
- Difference in purpose
  - Clustering finds the majority patterns in a data set and organizes the data accordingly
  - Outlier detection tries to capture those exceptional cases that deviate substantially from the majority patterns
- Difference in methodology
  - Outlier detection might also use supervision during the detection process
  - Clustering analysis is typically unsupervised in nature

34

Outliers versus Noise

- Noise is a random error or variance in a measured variable, and thus is not interesting in data analysis, including outlier detection
  - Example: a bigger lunch one day or one more cup of coffee than usual
- Outliers are interesting because they are suspected of not being generated by the same mechanism as the rest of the data
  - It is important to justify why the detected outliers are generated by some other mechanisms
  - Example: make various assumptions on the rest of the data and show that the detected outliers violate those assumptions significantly

35

Outlier Detection and Novelty detection

- Example: monitoring a social media web site where new content is incoming, novelty detection may identify new topics and trends in a timely manner
  - Novel topics may initially appear as outliers
- To this extent, outlier detection and novelty detection share some similarity in modeling and detection methods.
- Difference: in novelty detection, once new topics are confirmed, they are usually incorporated into the model of normal behavior so that follow-up instances are not treated as outliers anymore

36

- THIS MATER